

LeetCode 2658: Maximum Number of Fish in a Grid

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 22, 2026

Copyright: © 2026 Arpit Raj. All rights reserved.

Problem Intuition & Approach

The problem asks us to find the maximum amount of fish we can catch starting from any water cell. We can only traverse through adjacent water cells.

Algorithm (Depth-First Search):

- Scan the grid cell by cell. Treat every cell with fish ($\text{grid}[i][j] > 0$) as an unexplored water body.
- Initiate a **DFS (Depth-First Search)** from that cell to visit all connected water blocks.
- Keep a running total of the fish caught in the `curr` variable.
- **Crucial Step:** Mark visited cells by setting their value to 0 (turning them into land). This prevents infinite recursion and ensures we don't count the same fish twice.
- Keep tracking the maximum component sum in the `ans` variable.

Visual Walkthrough

Let's consider a simplified grid representation where numbers represent the amount of fish. Grey cells represent land (0).

Initial Grid State:



Finding Connected Components:



0	4	0	3
0	0	0	5

0	4	0	3
0	0	0	5

Execution Tracing:

- **Component 1 (Red):** Starting at the cell containing **2**, the DFS explores adjacent cells **1** and **4**.
Total = $2 + 1 + 4 = 7$ **fish**. After exploring, these cells are set to 0.
- **Component 2 (Blue):** The loop later hits the cell containing **3**, which connects to **5**.
Total = $3 + 5 = 8$ **fish**.

The global maximum returned by the algorithm is **8**.

Implementation Code (C++)

```
class Solution {
public:
    vector<pair<int,int>> dir={{-1,0},{1,0},{0,1},{0,-1}};
    int n;
    int m;

    void dfs(int r, int c, vector<vector<int>>& grid, int& curr){
        curr += grid[r][c];
        grid[r][c] = 0;

        for(auto it : dir){
            int nr = r + it.first;
            int nc = c + it.second;

            if(nr >= 0 && nc >= 0 && nr < n && nc < m){
                if(grid[nr][nc] > 0){
                    dfs(nr, nc, grid, curr);
                }
            }
        }
    }

    int findMaxFish(vector<vector<int>>& grid) {
        n = grid.size();
        m = grid[0].size();

        int ans = 0;
        for(int i = 0; i < n; i++){
            for(int j = 0; j < m; j++){
                if(grid[i][j] > 0){
                    int curr = 0;
                    dfs(i, j, grid, curr);
                    ans = max(ans, curr);
                }
            }
        }

        return ans;
    }
};
```